

Do Not Trust the Clouds Easily: The Insecurity of Content Security Policy Based on Object Storage

Yangzixing Lv, Wei Shi¹, *Member, IEEE*, Weiyong Zhang², Hui Lu³,
and Zhihong Tian⁴, *Senior Member, IEEE*

Abstract—The content security policy (CSP) is a World Wide Web Consortium (W3C) standard, designed to prevent and mitigate security vulnerabilities, such as cross-site scripting (XSS) attacks, data injection attacks, and clickjacking attacks on websites. In this article, we present a newly discovered front-end Web attack that uses the current object storage services vulnerability of cloud vendors to bypass CSP. We selected the object storage services from two cloud vendors with the most users, i.e., Google and Amazon, to conduct systematic and large-scale research and analysis. Three cyberspace search engines are used to retrieve data, from which we analyze the consequence and damage range of this security breach. We focus on reporting four key aspects of this security breach: 1) how to use object storage services to bypass CSP; 2) analysis on the existence of such vulnerability in real-world websites; 3) analysis on the existing security vulnerabilities in current object storage services; and 4) the new strategy on object storage services that we propose to use to eliminate the discovered security threat.

Index Terms—Content security policy (CSP), Web security.

I. INTRODUCTION

THERE are many front-end attacks on the Internet, such as cross-site scripting (XSS), cross-site request forgery (CSRF), data injection, and clickjacking [5], [22], [23], [32]. With the efforts made by numerous researchers and security companies, the detection [6], [7], [9] and defense methods [8], [10] on various attacks have greatly alleviated the cybersecurity vulnerabilities [11], [12]. However, not all vulnerabilities have been removed. With the continuous update and development of technology [17], [18], [19], many existing and new vulnerabilities are active on Internet [13], [14], [24].

Take XSS as an example: during an XSS attack, the attacker injects scripts into normal Web content and executes malicious code consequently. Given the browser cannot distinguish between normal Web code and malicious code, the attacker inherits the privileges of normal webpages through

XSS vulnerabilities and carry out the next attack. XSS is considered as one of the most dangerous attack in front-end security, because it had appeared on almost every existing website before the emergence of content security policy (CSP) [15], [20]. Before that, the most effective way to prevent XSS is to filter user input. This prevents dangerous content, such as malicious scripts, from being embedded into normal webpages. Unfortunately, filtering user input cannot completely prevent the input with malicious scripts. This is because the World Wide Web Consortium (W3C) keeps adding new tags and events to the HTML specification [38], which allows attackers to use them to bypass some input checks [39]. This led to XSS remaining to be one of the most common security problems on the Web, until the emergence of CSP. CSP is by far the most popular and mature XSS vulnerability solution, supported by W3C and all mainstream browsers. CSP successfully reduces the occurrence of XSS attacks and increases the cost of XSS attacks by prohibiting inline-script execution and restricting the script sources [31]. Furthermore, CSP can eliminate the risk of having strings converged into malicious code by banning `evil()` functions. A very strict CSP can almost eliminate XSS vulnerabilities.

As cloud computing gains its popularity, a large number of infrastructures gradually adopt the services provided by cloud vendors to reduce operation and maintenance costs. For example, developers of small and medium-sized enterprises now prefer to rent resources provided by cloud vendors instead of building their own IT infrastructure [35], [36], [37]. The cost of the latter is several times or even tens of times that of the former. Object storage is a low cost, high volume, and efficient cloud storage service provided by cloud vendors. It is mainly used to store unstructured data, such as images, audio, video, text, slides, etc. It is important to note that cloud vendors provide a default URL for each object stored on the object storage service to facilitate its users access to this object. The object storage service does not use the traditional tree structure. Instead, it only allows simple access based on buckets, and in-bucket modification on an object is not allowed.

A. Goals and Contributions

In this article, our main goal is to demonstrate an attack that we discovered on the front-end of websites. This attack uses cloud vendors' object storage services to bypass CSP. Through this article, we hope to bring this to the attention of website developers. Furthermore, we studied the security strategies in the object storage services of existing cloud vendors, these

Manuscript received 12 October 2022; revised 21 December 2022; accepted 13 January 2023. Date of publication 20 January 2023; date of current version 7 June 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62272114 and Grant U20B2046; in part by the National Key Research and Development Program of China under Grant 2021YFB2012402; and in part by the Guangdong Province Universities and Colleges Pearl River Scholar Funded Scheme (2019). (Corresponding authors: Hui Lu; Zhihong Tian.)

Yangzixing Lv, Weiyong Zhang, Hui Lu, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: 2112006173@e.gzhu.edu.cn; foxdream@163.com; luhui@gzhu.edu.cn; tianzhihong@gzhu.edu.cn).

Wei Shi is with the School of Information Technology, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: wei.shi@carleton.ca).

Digital Object Identifier 10.1109/JIOT.2023.3238658

strategies are mainly used to detect the origin of the request. Our research results show that the existing security strategy has security vulnerabilities. We then propose a new security strategy that implements a “double protection” mechanism jointly constructed by CSP and cloud vendors.

In addition, to better understand the impact range of this front-end security vulnerability, we report the analysis results on the website that contains the object storage root domain in the CSP whitelist. To perform the analysis, we selected the object storage services from two cloud vendors with a large number of users. We found out that no matter which cloud vendor’s object storage service a website uses, most fell in this front-end security vulnerability. For example, tens of millions of websites include Google Cloud’s object storage root domain in their CSP whitelist. More than 95% of these websites have this front-end security vulnerability. Finally, we conduct an in-depth analysis on Google Cloud. We found out that Google uses a “shared domain mechanism” and believe that it brings front-end security vulnerability to the developer’s website while offering the developers convenience. In addition, we analyzed the existing security strategies of the object storage services of two cloud vendors. Our results suggest that the existing security strategies cannot prevent such security threat. Therefore, we propose a new security strategy for object storage that can effectively mitigate this front-end security risk.

B. Structure

The remainder of this article is organized as follows. Section II introduces the basics of CSP and object storage. Section III describes the details of this new attack method. Section IV explains the data analysis method of this article and shows the data analysis results in the real world. Section V reports the reasons for the results of the data analysis. Section VI proposes a new security strategy of object storage. Section VII describes the work related to CSP security. Finally, Section VIII summarizes this article and puts forward prospects.

II. BACKGROUND

A. Content Security Policy

CSP is a series of rules, it is based on the whitelist mechanism. The browser will limit the resources loaded by the webpage according to the whitelist of the CSP, the resources that do not appear in the whitelist of CSP directives [21], [28], [29] will not be loaded into the webpage by the browser. For example, the CSP directive `img-src https://google.com` restricts webpages to only load image resources under the `google.com` domain, and the image resources must also use the HTTPS protocol. Images from any other sources will not be displayed on the webpage. CSP is a defense mechanism relying on the browser, which is often referred to as the firewall on the browser [29], [30]. The CSP whitelist is made by the website developer, and it takes effect by setting the HTTP/S response header or setting the meta tag.

CSP mainly narrows the attack space of front-end security vulnerabilities by restricting resources and increases the attack

TABLE I
DIRECTIVES WITH SECURITY CONTROL ABILITIES IN CSP

Directives	Directive describe
default-src	serves as a fallback for the other CSP directives
script-src	specifies valid sources for JavaScript
object-src	specifies valid sources for the object, embed, and applet elements.
frame-src	specifies valid sources for nested browsing contexts loading
child-src	specifies the valid sources for web workers and nested browsing contexts

cost of vulnerabilities. CSP provides defense directives against several vulnerabilities.

- 1) The `script-src` directive can restrict the loading of javascript script sources and XSLT script sources on webpages, and through the `unsafe-eval` and `unsafe-inline` keywords to limit the ability of scripts in webpages. In addition, you can also set `object-src` to limit the flash resources.
- 2) The directives `frame-src` and `child-src` limit the frame source of the website loading to prevent clickjacking attacks. Clickjacking attacks can cover a layer of invisible frame on the webpage. When a user clicks, what she/he actually clicked is the invisible frame of the attacker.

According to the CSP Level 3 document [40], we list several CSP directives that can control security vulnerabilities, as shown in Table I.

It is worth noting that there is a special directive in CSP, `default-src` directive serves as a fallback for the other CSP directives. If there is no whitelist for other directives, the browser will get resources based on the whitelist of `default-src`. But if other directive specifies a whitelist, `default-src` will be set to invalid state by the browser. The following example can illustrate this feature very well: (Note: “self” means the same source as the current pages)

```
Content-Security-Policy:
default-src https://google.com;
script-src 'self';
```

The above CSP is equal to the following CSP:

```
Content-Security-Policy: script-src 'self';
connect-src https://google.com;
img-src https://google.com;
object-src https://google.com;
frame-src https://google.com;
```

`Script-src` directive specifies valid sources for JavaScript. This includes not only URLs loaded directly into `<script>` elements but also things like inline script event handlers(`onclick`) and XSLT stylesheets which can trigger script execution. It is obvious that the `script-src` directive has the ability to restrict the script execution of the webpage. Therefore, `script-src` is mainly used to fight against the evil XSS vulnerabilities. By default, `script-src` directive has very strict restrictions on scripts. If one needs to enable special permissions for scripts, one must set keywords in the `script-src` directive. For example, if one needs to execute script inline functions in webpages, one must use keyword “`unsafe-inline`.” If one needs to convert a

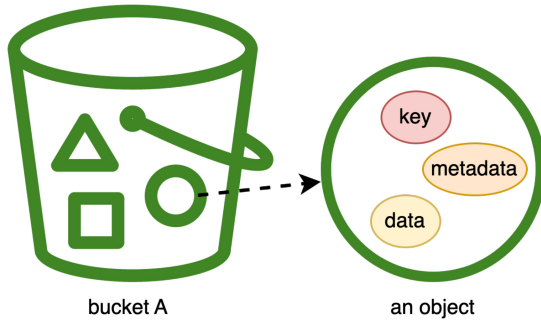


Fig. 1. Simple structure of object storage.

TABLE II
DETAILED EXPLANATION OF THIS URL LINK

URL Field	Explanation
xiaogui	“Bucket” name, unique identification ID
s3	AWS object storage service
ap-northeast-2	Geographic location ID of distributed storage
amazonaws.com	default root domain provided by AWS
24479C70-9EEF-4645-BFD4-6D0BCC88E8F1	Object file name
.jpeg	Object file type

string into an executable script in the webpage, one must use the command “unsafe-eval.”

All major browsers support CSP and more and more developers also start to apply CSP to protect the front-end security of their websites. The role of CPS currently appears to be very effective: a correct CSP can effectively defend against XSS or clickjacking attacks when developers make programming mistakes. However, developers need to be aware that for the front-end security of the website, they should use CSP to limit the capabilities of the script to a minimum.

B. Object Storage

Object storage is a “bucket” storage structure provided by cloud vendors that can store any type of file. Object storage does not have a directory tree, and users PUT and GET objects in the bucket of the object storage. Fig. 1 is the structure of a simple object storage.

Each object is mainly composed of three parts: 1) key is the unique identifier of this object; 2) data is the data content of this object; and 3) metadata is the attribute of this object, such as file type, size, date, etc. They are mainly used for searching and sorting objects. Take AWS S3 storage as an example, if the properties of an object are set to public, then AWS S3 will assign a default URL link for this object, and anyone can access this object through this URL link. The following is a link to an image object stored on AWS S3, and the detailed explanations as shown in Table II.

<http://gzhu.s3.ap-northeast-2.amazonaws.com/24479C70-9EEF-4645-BFD4-6D0BCC88E8F1.jpeg>

Each bucket has a unique identification ID. Therefore, object storage services of a same cloud vendor are not allowed to have buckets with the same name. It is worth noting that the default root domain “amazonaws.com” is provided by AWS S3. The default root domain of all public objects stored

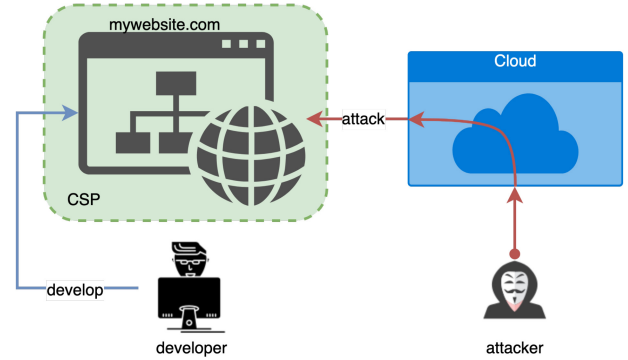


Fig. 2. Threat model.

on AWS S3 is “amazonaws.com.” This feature is very important and plays an important role in the new attack methods shown in this article. *Note:* In order to facilitate the reader’s understanding of the context, the “default root domain,” “root domain,” and “object storage root domain” in this article all have the same meaning.

III. NEW ATTACK METHOD

In this section, we introduce a way to use object storage to bypass CSP and attacks a website. This security vulnerability is essentially a trust issue: the website developers trust cloud vendors too much, causing the developers make mistakes in the CSP configuration. Fig. 2 illustrates the proposed threat model.

As shown in Fig. 2, attackers uses the cloud to complete her/his attack tasks. Taking the object storage service as an example, all cloud vendors provide a URL link to access the public objects in the object storage “bucket,” and the root domain of the URL link provided by the same cloud vendor stays the same. After some website adopt object storage services to store the unstructured data of the website, the developer puts the root domain of the URL link provided by the cloud vendor into the CSP whitelist. Unfortunately, many developers over trust the cloud providers, consequently, choose to put the object storage root domain into the default-src of the CSP whitelist, or even worse, into the script-src. Because the script-src has the highest authority on a webpage. This misconfiguration causes CSP to be ineffective. When the security vulnerabilities are caused by such developers errors, CSP can no longer be able to defend against attacks as intended.

Take the most harmful front-end attacks, XSS as an example, we now explain how to attack websites using object storage. Earlier in this article, we have mentioned that both default-src and script-src have the ability to limit the script execution of the front-end of a webpage. In the case of the CSP configuration error in the webpage, attackers can use object storage to bypass CSP script restrictions and execute malicious javascript codes on webpages to complete XSS attacks. Fig. 3 shows the traditional XSS fails to attack in the presence of the CSP protection.

We can see from Fig. 3 that an XSS attack appears in mywebsite.com page. The attacker wants to exploit this XSS vulnerability to execute malicious javascript code in the

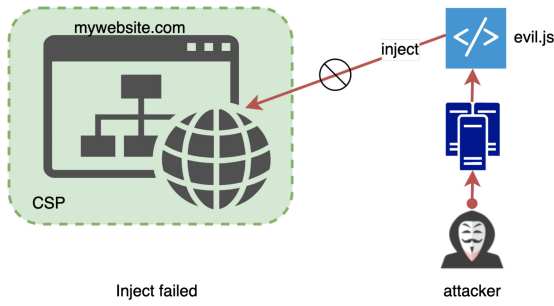


Fig. 3. Failed attack.

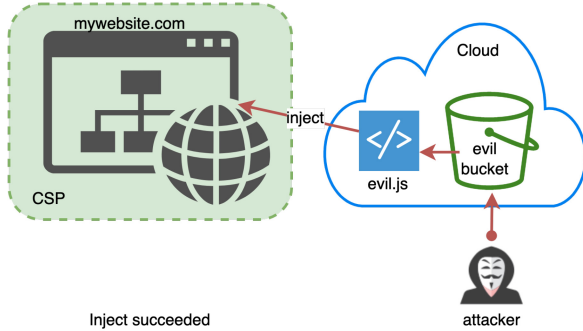


Fig. 4. Attack the website with object storage.

mywebsite.com webpage. The attacker sets up a server called “evil.com” and created a malicious script evil.js. Then, the attacker uses the XSS vulnerability to inject `<script>` to load the evil.com/evil.js script into the webpage. However, because the evil.com is not in the whitelist of script-src directive, the browser will prohibit mywebsite.com webpages from loading evil.com/evil.js. Therefore, the attacker’s attempt to exploit the XSS to attack the website fails. Fig. 4 illustrates how an attacker carries out a successful attack on a website using the object storage service to bypass the CSP protections.

As shown in Fig. 4, traditional XSS attacks are blocked by CSP protections. Then, the attacker finds out that the object storage root domain appears in the whitelist of script-src, the attacker can use the cloud to carry out XSS attacks. The attacker uploads evil.js script to the bucket of the cloud vendor’s object storage. As the root domain provided by a cloud vendor is unified, and this root domain is in the CSP whitelist of mywebsite.com, the browser will not block requests to this object storage domain. Therefore, the attacker uses an XSS to inject evil.js into the webpage, an external malicious script in the webpage can be successfully executed to complete the attack. Similarly, any other attack relies on external malicious scripts can be carried out using this object storage service. Such bypass approach leaves CSP to be totally ineffective.

Fundamentally, it is the trust the developers have on cloud service providers and the convenience they want to open the door for exploitation by the attacks. We would like to point out that what we present in this article has something in common with the attack methods mentioned by Google researchers [2]: both attacks rely on a third party to complete the attack. However, the attack method mentioned in this article only requires the creation of a bucket in the cloud

TABLE III
AMOUNT OF DATA

Data Sources	Google Object Storage Root Domain “googleapis.com”	Amazon Object Storage Root Domain “amazonaws.com”
Fofa	10,374,238	258,360
Quake	7,472,832	840,083
Zoomeye	11,224,359	2,025,326

vendor’s object storage service that enables the upload of malicious files and can easily bypass the CSP protections. On the other hand, the method proposed by Google researchers [2] rests on exploiting the vulnerabilities of third-party resources, for example, user-controllable callback [25], [26] or mistaken HTTP response [27], etc. Although both attack methods are equally effective for bypassing CSP, it is clear that the attack method proposed in this article is simpler.

IV. REAL-WORLD DATA ANALYSIS

A. Methodology

In this section, we discuss how this new attack exists in real world and how much risk it imposes. We noticed that many websites have set the root domain of the object storage service in the CSP whitelist. Following the introduction of CSP in Section II, we collect the deployment of three CSP directives. If these three directives are bypassed, we deem this website to be vulnerable.

- 1) *script-src* and *object-src*: These two CSP directives are mainly used to prevent XSS from loading external evil resources, script-src mainly prevents attackers from loading evil javascript code, object-src is mainly used to prevent flash type XSS.
- 2) *frame-src* and *child-src*: These two CSP directives are mainly used to limit the external frame resources that the webpage can load, So they all have the ability to defend against clickjacking vulnerabilities.
- 3) *default-src*: This directive is special in CSP, as mentioned in Section II of this article, this directive takes effect when other directives do not specify a whitelist. It means that this directive can act as any directive under special circumstances, Therefore, default-src can defend against XSS or clickjacking.

We detect the website’s CSP whitelist based on the above-mentioned three security scenarios. We check whether the root domain of the object storage service appears in the whitelist of these CSP directives and analyze whether these CSP directives can be bypassed by the object storage root domain to evaluate the security of these websites. We used these three cyberspace search engines to collect data: 1) Fofa; 2) Quake; and 3) Zoomeye. The main collection targets are those websites that have included the object storage root domain in their CSP whitelist. Our main research target is Google and Amazon. Table III shows the amount of data collected by three cyberspace search engines.

These three pieces of data are partially duplicated, so we will not combine them, instead use them for comparative analysis. We extracted the content of the CSP whitelist of these websites and split them according to different directives of

TABLE IV
TOP TEN CSP DIRECTIVES WITH OBJECT STORAGE ROOT DOMAIN

<i>Whitelist contains googleapis.com</i>	<i>Whitelist contains amazonaws.com</i>
1.Style-src	1.Img-src
2.Script-src	2.Connect-src
3.Img-src	3.Default-src
4.Font-src	4.Style-src
5.Default-src	5.Script-src
6.Connect-src	6.Media-src
7.Media-src	7.Font-src
8.Object-src	8.Object-src
9.Form-src	9.Child-src
10.Script-src-elem	10.Script-src-elem

the CSP. We first extract the Content-Security-Policy field of an http response packet, and then match and split the content of the Content-Security-Policy field through regular expressions according to the CSP specification [40]. The specific data analysis is shown in Section IV-B. We then check these whitelists based on the three risky CSP directives. In the process of detecting the whitelist of CSP directives, we focused on the following three issues.

- 1) Default-src only takes effect when no other directives are specified in the CSP, therefore, in the detection process, we must first check whether the CSP specifies other risk directives (script-src, object-src, frame-src, and child-src). Otherwise, even if the object storage root domain is included in the whitelist of default-src, it will not take effect. In this case, the CSP whitelist setting is correct and cannot be bypassed.
- 2) When developers add the object storage root domain to the CSP whitelist, it may contain the geographic location ID of the object storage. In this case, the attacker can still choose the same storage location to bypass the CSP whitelist. However, some website developers also write the unique identifier of the object storage bucket (the bucket name) into the CSP whitelist, and the object storage services of a cloud vendor are not allowed to have the same bucket name. In this case, the attacker cannot use object storage to bypass the CSP. In this case, the CSP whitelist setting is correct and cannot be bypassed.
- 3) The developers often make spelling mistakes when setting the CSP whitelist, such as the object storage location ID “ap-northeast-2” as introduced in Section II of this article. During the process of detecting the CSP whitelist, we found that a small number of developers incorrectly spelled it as “ap.northeast.2.” In this case, the CSP whitelist setting is correct and cannot be bypassed.

B. Real-World Data

Based on the above issues, we perform statistical analysis on the collected data. We found that developers often add the object storage root domain into the whitelist of multiple CSP directives. Table IV shows the top ten CSP directives of the object storage root domain in the whitelist.

From Table IV, we can see that the top five directives that contain the root domain of Google or Amazon object storage are almost the same. This is related to the purpose of object storage: object storage services are mainly used to store

TABLE V
RANK OF DIRECTIVES

The rank of directives whitelist contains the root domain of Google and Amazon's		
Directives	Number of websites	Percentage of total
1.default-src	2079583	80.81%
2.img-src	1888224	5.41%
3.script-src	1534049	4.52%
4.style-src	1036501	3.05%
5.connect-src	506524	29.90%

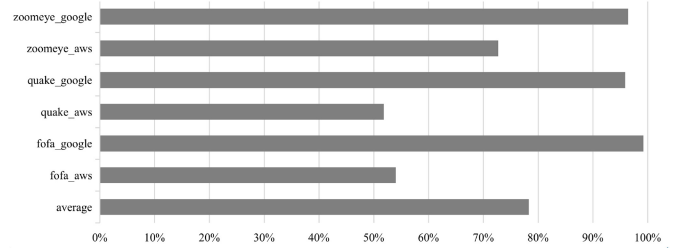


Fig. 5. Percentage of websites with incorrect CSP.

unstructured data. Therefore, img-src and media-src directives are ranked higher. But what caught our attention is that the top Google rank is the style-src and script-src. This is related to the “shared domain mechanism” adopted by Google, which will be mentioned in Section V of this article. In addition, half of the directives in this rank are security risks, such as script-src, default-src, object-src, script-src-elem can limit the script resources, and child-src can limit the frame resources. We found that the object storage root domain appeared in multiple CSP directives of these websites. We counted the directives for both Google and Amazon root domains in the whitelist. Table V shows the specific data of the top five directives.

Our statistical analysis of the whitelist of CSP directives on these websites is based on the proposed risk model. We found that the CSP of most of these websites can be bypassed. On average, the configuration error rate of these websites with object storage root domains in the whitelist is as high as 78.3%. Fig. 5 illustrates the ratio of the websites with incorrect CSP to the total number.

We analyzed these CSP directives that can be bypassed and classified according to the Google and Amazon, reflect the differences between the users of the two companies. Fig. 6 shows the CSP directives that can be bypassed by the attackers. Google’s object storage root domain is mainly included in the whitelist of script-src and default-src. The two directives together account for more than 99.5%. It appears less frequently in the whitelist of the other risk directives. In contrast, Amazon’s object storage root domain distribution is more diverse, it appears in the whitelist of all risk directives. This difference is related to the different business models adopted by Google and Amazon. This difference is explained in Section V. We have counted the geographic locations of websites that these CSPs can be bypassed. We found that websites with misconfigured CSP are mainly concentrated in countries with developed Internet. This may be related to the

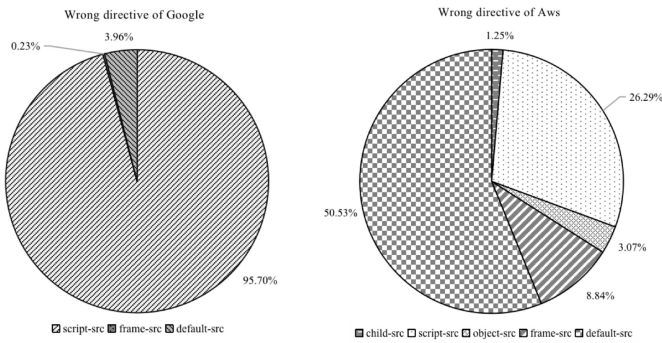


Fig. 6. Directives can be by passed.

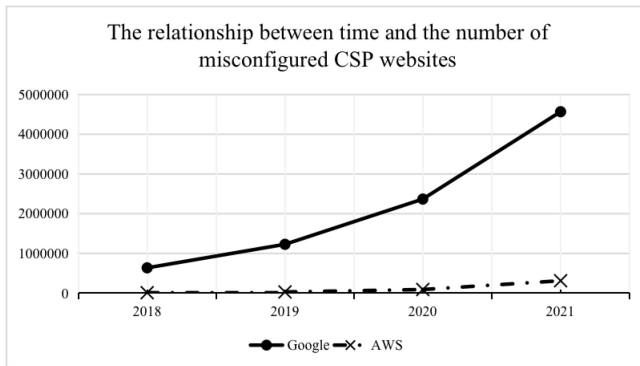


Fig. 7. Number of websites and time.

TABLE VI
REGIONAL RANK OF WEBSITES

Location of CSP misconfigured website	
Amazon	Google
1.US	1.China
2.Ireland	2.Germany
3.Netherlands	3.US
4.Australia	4.France
5.Germany	5.South Korea
6.U.K	6.Netherlands
7.Japan	7.Spain
8.China	8.U.K
9.Brazil	9.Australia
10.Singapore	10.Switzerland

number of Internet users and developers in the region. The regional rank is shown in Table VI.

In addition, we have counted the time when websites misconfigured with the CSP whitelist appeared. These time data are the time when those cyberspace search engines collected a website. We found that the number of websites with incorrectly configured CSPs has been increasing year by year. As shown in Fig. 7, we can see that the number of websites with Google's object storage root domain present in the CSP whitelist doubles every year. However, the number of websites with Amazon's object storage root domain in the CSP whitelist has risen slowly. We speculate that in the future, there will be more websites with cloud vendor object storage root domains in their CSP whitelist.

Through analysis of the real-world data, we demonstrate in this article the coverage and impact of this security vulnerability. We believe that it is insecure to include the object storage

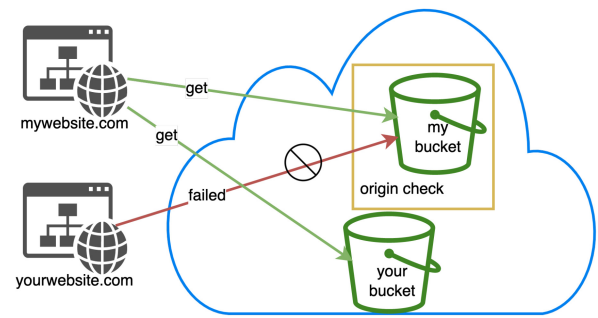


Fig. 8. CORS security strategy.

root domain in the CSP whitelist, it makes the CSP's ability to protect a website completely ineffective.

V. ANALYSIS OF CAUSES

In this section, we will explain the reasons for the real-world data analysis results in Section IV. We will analyze the framework of object storage and the special mechanism of Google Cloud.

A. Analysis of Framework

To analyze the reasons for this security vulnerability on the side of the cloud vendors, we studied the security strategies of Google and Amazon's object storage. According to our research, there is only one security strategy of Google and Amazon object storage services.

- 1) *Cross-Origin Resource Sharing (CORS)*: Developers can set cross-origin rules for object storage "bucket." By detecting the origin of the request, the developer can prohibit other websites from accessing his "bucket." Fig. 8 shows a simplified diagram of this CORS security strategy. As shown in Fig. 8, after the developer has configured the origin check rule of "my bucket," Requests from "yourwebsite.com" will not be able to get the objects stored in "my bucket." But "mywebsite.com" can still access "your bucket" that is not configured with origin check rules.

According to our research, we found that cloud vendors have a flaw in security strategy. This flaw is the reason for this new attack method introduced in this article to bypass CSP. From Fig. 8, we can see that even if the developer of "mywebsite.com" sets the origin detection rule, "mywebsite.com" can still access the resources in "your bucket." This security strategy may be risky. If there are security attacks such as XSS or clickjacking in the webpage of "mywebsite.com," attackers can use object storage services to bypass the CSP whitelist and load external evil resources. Fig. 9 illustrates the attack model.

We believe that the existing security strategies of cloud vendors's object storage are not fully effective in protecting developers' websites. To solve the flaw in the security strategy of cloud vendors's object storage services, we have improved a new origin security strategy based on the old strategy. This new strategy can effectively solve the risk of CSP being bypassed by object storage.

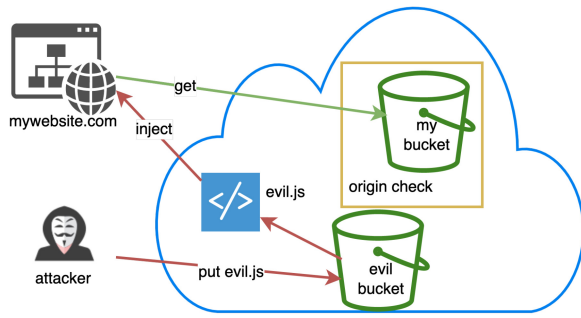


Fig. 9. Attack model.

TABLE VII
SUBDOMAINS AND RESOURCES

Root domain: googleapis.com	
Business	Subdomain
Public Css Resources	fonts.googleapis.com
Public Js Resources	ajax.googleapis.com
Google Mail API	mail.googleapis.com
Google Map API	maps.googleapis.com
Google Translate API	translate.googleapis.com
Google Object Storage	storage.googleapis.com

B. Uncontrolled Domain Sharing

In this section, we will explain to readers why Google's risk coverage in Fig. 5 is over 95%. After our investigation and analysis, we found that the root domain of the object storage provided by Google has this characteristic: the objects storage root domain is shared with other business root domains. We investigated the root domain "googleapis.com" provided by Google Cloud Object Storage Service, we found that Google provides many public APIs or static resources on these subdomains, most of which are front-end static resources or business API. Table VII lists some common subdomains and resources.

According to the public information [33], root domain "googleapis.com" has at least 2000 public subdomains. We further discovered a problem on Google's object storage service: the objects storage shares the root domain with other businesses. This uncontrolled sharing domain mechanism may cause a lot of security vulnerabilities, especially the threats on the front-end security of websites.

It is undeniable that these APIs or static resources provided by Google are very beneficial to the developers. Because the resources provided by Google are very effective and convenient, many developers use the resources provided by Google to help them develop their websites. Unfortunately, they set the CSP whitelist of their websites with Google provided root domains, give these resources the highest authority, and configure these API domains in the whitelist of script-src or default-src. The developers believe that these domains are only controlled by Google, therefore, they are all safe. However, after Google adopted the "shared domain mechanism," these domains have become less secure and trustworthy.

Fig. 10 illustrates the risk model. When developers trust the API or static resources provided by Google, developers also trust storage.googleapis.com. But storage.googleapis.com is the domain of Google Cloud's object storage service, anyone can store any file under storage.googleapis.com. There is an exploitation of trust here: the attackers can upload malicious files in the "evil bucket" of the object storage service and

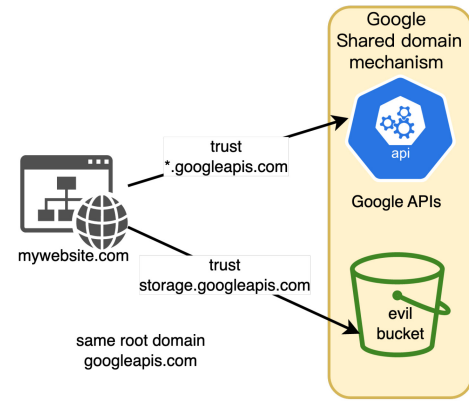


Fig. 10. Shared domain mechanism.

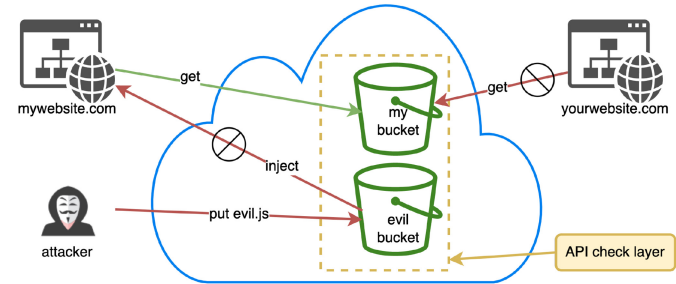


Fig. 11. New security strategy.

use the root domain provided by Google to bypass the website's CSP whitelist. According to our statistics, script-src: *.googleapis.com appears in the CSP of more than ten million websites. This misconfiguration destroys the effectiveness of CSP. We believe that Google's shared domain mechanism is very unsafe.

VI. CLOUD API CHECK LAYER

To solve the flaws in the existing object storage security strategy mentioned in Section V, we make improvements on the origin security strategy. We propose a new security strategy to eliminate the vulnerability on CSP bypass. Fig. 11 is a simplified diagram explaining our proposed security strategy.

As shown in Fig. 11, a cloud vendor's API check layer is added on top of the origin security strategy. The "my bucket" created by a developer can be bound to the website "mywebsite.com." After this binding, yourwebsite.com will not be able to access the resources in "my bucket." Compared to the original security strategy, our proposed new strategy insures the API check layer of cloud vendors will indeed check all requests whose origin is mywebsite.com and prohibit it from accessing any other storage "buckets" except for "my bucket" that has the binding relationship completed. After implementing this binding relationship, it is impossible to bypass the CSP whitelist with object storage service.

Let us assume an attack scenario. Now, on "mywebsite.com" There is an XSS vulnerability in a webpage. An attacker can inject "evil.js" into the webpage to execute malicious javascript by using the "evil bucket." However, under the protection of the new security strategy, the webpage of "mywebsite.com" only allows to load the resources in "my bucket," and the resources in "my bucket" are controlled by

the developer. Therefore, the attacker is prevented from using XSS to execute malicious javascript.

We believe that this API check layer can provide a “double protection” security mechanism of “CSP and object storage” for the front end of the websites. The developers can creating the binding relationship between a website and a specific object bucket, which in turn, forbids this website from requesting resources in other “bucket.” This “double protection” security mechanism can effectively prevent attackers from using object storage services to bypass CSP.

VII. RELATED WORK

Many security researchers have studied the security of CSP. In the research work of the paper [2], the author established a CSP threat model and proposed several CSP error cases based on the threat model. The author also summarizes the shortcomings of the CSP whitelist and ways to bypass the CSP whitelist, such as user-controllable callback, mistaken HTTP response, etc. The author also analyzes the CSP of the real-world website through the Google search engine, he pointed out that the CSP deployment of most of the websites is invalid. In addition, the author proposed the “strict-dynamic” idea, which helps to create a strategy based on encrypted nonces instead of relying on a domain whitelist. Both the paper and this article point out that the CSP whitelist mechanism is not safe and the attackers can bypass the CSP whitelist and attack the website.

In addition, the security researcher of CSP used a semantic analysis method to study CSP2.0 [1]. The main focus of this article is on the browser support of CSP2.0, semantic analysis of CSP2.0, and deployment of CSP2.0. Although the author mentioned in his paper the security issues that are prone to CSP, it is only a statistical analysis of the “unsafe-inline” and “unsafe-eval” keywords in script-src and default-src.

In an earlier research [3], the author raises the security issues caused by dynamically generated scripts, mainly because the callback parameter in the jsonp interface is user-controllable. In order to combat the risks brought by similar jsonp interfaces, the author recommends not to whitelist the source of dynamic scripts. Therefore, this article proposes a script template mechanism that allows the developers to separate dynamic data values from static code.

Unfortunately, according to Weissbacher et al.’s 2014 study [28], only 1% of the top 100 webpages use CSP. Their research found that it was very difficult to deploy an initial CSP whitelist, because this would lead to a large number of changes to existing applications.

In the previous research work [4], Kerschbaumer et al. found that the insecure keyword “unsafe-inline” was used in the CSP of many webpages. Therefore, the author has created a system to collect legitimate scripts recognized by users and list the hashes of these legitimate scripts in the CSP white list to prevent developers from using contaminated open-source scripts.

VIII. CONCLUSION

In this article, we report an attack method that uses object storage services to bypass the CSP whitelist to carry out

attacks on webpages. We investigated on the object storage services of two cloud vendors and analyzed the flaws of cloud vendors’s existing object storage security strategies. We then proposed a new security strategy that will remove this security vulnerability.

Furthermore, we inspected over ten million of websites that adopted object storage services and decide on the existence of this vulnerability on these websites’ CSP whitelist configurations. Unfortunately, 78.3% of the CSP whitelists of the inspected websites have this security risks. According to our statistics, the CSP of at least 9 million websites currently is vulnerable to our proposed attacks. After bypassing the CSP protection, an attacker can inject external malicious resources into a webpage. In addition, because of Google provides a lot of public resources and adopts the “shared domain mechanism,” it opens the doors for security vulnerability.

The CSP protection capabilities are very important to the Web-development front-end security. CSP is still the most effective front-end security strategy on browsers so far. We believe it is very important that the CSP’s whitelist mechanism maintains its validity and correctness at all times. As a result, we propose a new object storage security strategy that works on CSPs and completes a “double protection” security mechanism. Even in the case that a developer grants the highest authority to the storage root domain in the CSP configuration, the front-end security of the website is guaranteed.

In future research work, we hope to apply this security strategy to real cloud services, rather than just existing in theory. In practical applications, we can improve this security strategy through real data to make it more efficient. We hope to work with more researchers to promote the security of CSP.

REFERENCES

- [1] S. Calzavara, A. Rabitti, and M. Bugliesi, “Semantics-based analysis of content security policy deployment,” *ACM Trans. Web*, vol. 12, no. 2, pp. 1–36, 2018.
- [2] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, “CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1376–1387.
- [3] M. Johns, “Script-templates for the content security policy,” *J. Inf. Secur. Appl.*, vol. 19, no. 3, pp. 209–223, 2014.
- [4] C. Kerschbaumer, S. Stamm, and S. Brunthaler, “Injecting CSP for fun and security,” in *Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2016, pp. 15–25.
- [5] CERT/CC, “Cert advisory CA-2000-02 malicious HTML tags embedded in client Web requests,” Feb. 2000. Accessed: Jun. 30, 2001. <http://www.cert.org/advisories/CA-2000-02.html>
- [6] N. Jovanovic, C. Krügel, and E. Kirda, “Pixy: A static analysis tool for detecting Web application vulnerabilities,” in *Proc. IEEE Symp. Secur. Privacy*, 2006, pp. 258–263.
- [7] S. Lekies, B. Stock, and M. Johns, “25 million flows later: Large-scale detection of DOM-based XSS,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 1193–1204.
- [8] Y. Nadji, P. Saxena, and D. Song, “Document structure integrity: A robust basis for cross-site scripting defense,” in *Proc. NDSS*, 2009.
- [9] P. Saxena, S. Hanna, P. Poosankam, and D. Song, “FLAX: Systematic discovery of client-side validation vulnerabilities in rich Web applications,” in *Proc. NDSS*, 2010.
- [10] M. T. Louw and V. N. Venkatakrishnan, “Blueprint: Robust prevention of cross-site scripting attacks for existing browsers,” in *Proc. 30th IEEE Symp. Secur. Privacy*, 2009, pp. 331–346.
- [11] D. Bates, A. Barth, and C. Jackson, “Regular expressions considered harmful in client-side XSS filters,” in *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 91–100.

- [12] D. Ross. "IE 8 xss filter architecture/implementation." 2008. [Online]. Available: <http://goo.gl/eOiPsI>
- [13] "Common vulnerabilities and exposures: The standard for information security vulnerability names." Accessed: Dec. 19, 2015. [Online]. Available: <https://cve.mitre.org/>
- [14] J. Grossman, "WhiteHat website security statistics report," WhiteHat, Santa Clara, CA, USA, White Paper, 2007.
- [15] S. Stamm, B. Sterne, and G. Markham, "Reining in the Web with content security policy," in *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 921–930.
- [16] H. Bojinov, E. Bursztin, and D. Boneh, "XCS: cross channel scripting and its impact on Web applications," in *Proc. 16th ACM Conf. Comput. Commun. Secur.*, Nov. 2009, pp. 420–431.
- [17] M. Heiderich, M. Niemietz, F. Schuster, T. Holz, and J. Schwenk, "Scriptless attacks: Stealing the pie without touching the sill," in *Proc. ACM Conf. Comput. Commun. Secur.*, Oct. 2012, pp. 760–771.
- [18] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, "mXSS attacks: Attacking well-secured Web-applications by using innerhtml mutations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 777–788.
- [19] A. Klein, "DOM based cross site scripting or XSS of the third kind," in *Proc. Web Appl. Security Consortium*, Apr. 2005, pp. 365–372.
- [20] M. West, A. Barth, and D. Veditz, "Content security policy level 2," W3C, Cambridge, MA, USA, Jul. 2015.
- [21] M. West, "Initial assignment for the content security policy directives registry," Google, Inc., Mountain View, CA, USA, RFC 7762, Jan. 2016.
- [22] R. Hansen and J. Grossman. "Clickjacking." Sep. 2008. [Online]. Available: <https://www.sectheory.com/clickjacking.htm>
- [23] M. Zalewski. "Postcards from the post-XSS world." 2011. [Online]. Available: <http://lcamtuf.coredump.cx/postxss>
- [24] Owasp top-10 2017. "OWASP foundation." Feb. 2017. [Online]. Available: <http://www.owasp.org.cn/>
- [25] M. Zalewski. "The subtle/deadly problem with CSP." 2011. [Online]. Available: <http://goo.gl/sK4w7q>
- [26] B. Hayak. "Same origin method execution(some): Exploiting a callback for same origin policy bypass." May 2015. [Online]. Available: <https://www.blackhat.com/docs/eu-14/materials/eu-14-Hayak-Same-Origin-Method-Execution-Exploiting-A-Callback-For-Same-Origin-Policy-Bypass-wp.pdf>
- [27] E. Homakov, "Using content-security-policy for evil." 2014. [Online]. Available: <http://homakov.blogspot.com/2014/01/using-content-security-policy-for-evil.html>
- [28] M. Weissbacher, T. Lauinger, and W. Robertson, "Why is CSP failing? Trends and challenges in CSP adoption" in *Proc. 17th Int. Symp. Res. Attacks Intrusions Defenses (RAID)*, Gothenburg, Sweden, Sep. 2014, pp. 212–233.
- [29] K. Patil and B. Frederik, "A measurement study of the content security policy on real-world applications," *Int. J. Netw. Secur.*, vol. 18, no. 2, pp. 383–392, 2016.
- [30] S. Liu, X. Yan, Q. Wang, and Q. Xi, "A systematic study of content security policy in Web applications," *Secur. Commun. Netw.*, vol. 9, no. 16, pp. 3570–3584, 2016.
- [31] M. Ying and S. Q. Li, "CSP adoption: current status and future prospects," *Secur. Commun. Netw.*, vol. 9, no. 17, pp. 4557–4573, 2016.
- [32] G. E. Rodríguez, J. G. Torres, P. Flores, and D. E. Benavides, "Cross-site scripting (XSS) attacks and mitigation: A survey," *Comput. Netw.*, vol. 166, Jan. 2020, Art. no. 106960.
- [33] Threatbook. "Subdomains of 'googleapis.com'" [EB/OL]. Nov. 2021. [Online]. Available: <https://x.threatbook.com/v5/domain/googleapis.com>
- [34] E. Kirda, N. Jovanovic, C. Kruegel, and G. Vigna, "Client-side cross-site scripting protection," *Comput. Secur.*, vol. 28, no. 7, pp. 592–604, 2009.
- [35] Y. Sun, Z. Tian, M. Li, S. Su, X. Du, and M. Guizani, "Honey-pot identification in softwarized industrial cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5542–5551, Aug. 2021.
- [36] Y. Sun, Z. Tian, M. Li, C. Zhu, and N. Guizani "Automated attack and defense framework toward 5G security," *IEEE Netw.*, vol. 34, no. 5, pp. 247–253, Sep./Oct. 2020.
- [37] Y. Ren, Y. Xiao, Y. Zhou, Z. Zhang, and Z. Tian, "CSKG4APT: A cybersecurity knowledge graph for advanced persistent threat organization attribution," *IEEE Trans. Knowl. Data Eng.*, early access, Jul. 20, 2022, doi: [10.1109/TKDE.2022.3175719](https://doi.org/10.1109/TKDE.2022.3175719).
- [38] World Wide Web Consortium (W3C). "HTML and DOM specifications[EB/OL]." May 2019. Accessed: Nov. 2021. [Online]. Available: <https://www.w3.org/html/>
- [39] PortSwigger. "Cross-site scripting (XSS) cheat sheet[EB/OL]." Jan. 2022. Accessed: Mar. 2022. [Online]. Available: <https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>

- [40] World Wide Web Consortium (W3C). "Content security policy level 3[EB/OL]." Oct. 14, 2022. Accessed: Oct. 2022. [Online]. Available: <https://w3c.github.io/webappsec-csp/>



Yangzixing Lv is currently pursuing the M.S. degree with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China.

His research interests include cyberspace security, cloud security, and data analysis.



Wei Shi (Member, IEEE) received the Bachelor of Computer Science and Technology degree from Harbin Institute of Technology, Harbin, China, in 2001, and the master's and Ph.D. degrees in computer science from Carleton University, Ottawa, ON, Canada, in 2003 and 2007, respectively.

She is a Professor with the School of Information Technology, Faculty of Engineering and Design, Carleton University. She is specialized in algorithm design and analysis in distributed systems, such as distributed data center and clouds, edge network,

mobile agents and actuator systems, and wireless sensor networks. She has also been conducting research in data privacy and big data analytics. She has published over 100 articles in top-ranked conferences and journals, such as IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON CLOUD COMPUTING, *ACM Transactions on Internet of Things*, and IEEE/ACM TRANSACTIONS ON NETWORKING.

Dr. Shi is also a Professional Engineer licensed in Ontario, Canada.



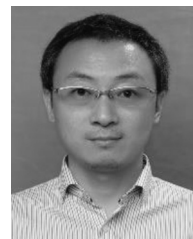
Weiyong Zhang is currently pursuing the Ph.D. degree with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China.

His current research interests include cloud-native security and cyberspace security.



Hui Lu received the B.E. degree in electronic science and technology and the M.S. degree in optical engineering from Southwest Jiaotong University, Chengdu, China, in 2003 and 2006, respectively, and the Ph.D. degree in electromagnetic field and microwave technology from Beijing University of Posts and Telecommunications, Beijing, China, in 2010.

He is a Professor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. He was a Research Associate with the Institute of Microelectronics, Chinese Academy of Sciences, Beijing, from 2010 to 2017. In 2017, he joined Guangzhou University. His research interests include cyberspace security and intelligent attack–defense.



Zhihong Tian (Senior Member, IEEE) received the B.E. and Ph.D. degrees in computer science and technology from Harbin Institute of Technology, Harbin, China, in 2001 and 2006, respectively.

He is currently a Professor and the Dean of the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, Guangdong, China. He is also a Guangdong Province Universities and Colleges Pearl River Scholar (Distinguished Professor). He is also a part-time Professor with Carleton University, Ottawa, ON, Canada. Previously, he served in different academic and administrative positions with Harbin Institute of Technology. His research interests include computer networks and cyberspace security. He has authored over 200 journal and conference papers in the above areas.